Developer Full Stack Test

Blossom

Objective

Evaluate the candidate's skills in developing full stack applications, using frontend and backend technologies such as React 18, GraphQL, Express, relational databases, cache management, and unit testing.

Description:

The candidate will develop a full stack application that allows for searches of characters from the Rick and Morty series, using the public Rick and Morty API. The application should have a user interface that enables viewing and managing characters, and a backend that handles queries and caching.

Frontend Requirements:

- Create a frontend application using React 18, GraphQL, and React Router DOM.
- The application should list the characters from the API in cards with their name, image, and species.
- Implement sorting of the characters by name (A-Z / Z-A).
- Display the details of each character when clicking on a card, including the image and the option to mark them as favorites.
- Implement the ability to add comments to the characters.
- The application must be responsive and make proper use of CSS Flexbox and CSS Grid.
- Use TailwindCSS for the styles.
- (Optional) Use TypeScript for the project development.
- (Optional) Implement soft-delete for characters.
- (Optional) Implement search by: Status, Species, and Gender.
- (Optional) Implement unit tests for at least 3 components or layouts.

Backend Requirements:

- Create an API using Express and GraphQL to allow searching for characters from Rick and Morty.
- Implement queries that allow filtering characters by Status, Species, Gender, Name, and Origin.
- Connect the API to a relational database using Sequelize, with configurations via migrations (e.g., MySQL or PostgreSQL).
- Implement connection to Redis to cache search results.
- Initialize the relational database with 15 characters from the Rick and Morty API.
- Implement middleware that prints relevant information for each request to the console.
- (Optional) Create a cron job that runs every 12 hours to update characters in the database.
- (Optional) Implement a method decorator that prints the execution time of queries to the console.
- (Optional) Implement unit tests for the character search query.
- (Optional) Use TypeScript for project development.
- (Optional) Apply software design patterns.

Deliverables:

- Project source code in a public GitHub repository.
- ERD diagram of the database.
- Documentation explaining how to run the application and how to use the API (can be a README file in the repository or a Wiki in the same repository).
- (Optional) Swagger documentation for consuming the API.

Evaluation Criteria:

- Compliance with the requirements.
- Proper use of the mentioned technologies.
- Code quality (structure, readability, comments, etc.).
- Visual appearance and usability of the application.
- Proper handling of Git.

Resources:

- API de Rick and Morty
- Link a Figma: Playground-Figma
- Link de Sequelize

Thank you and good luck

blossom

www.blossom.net